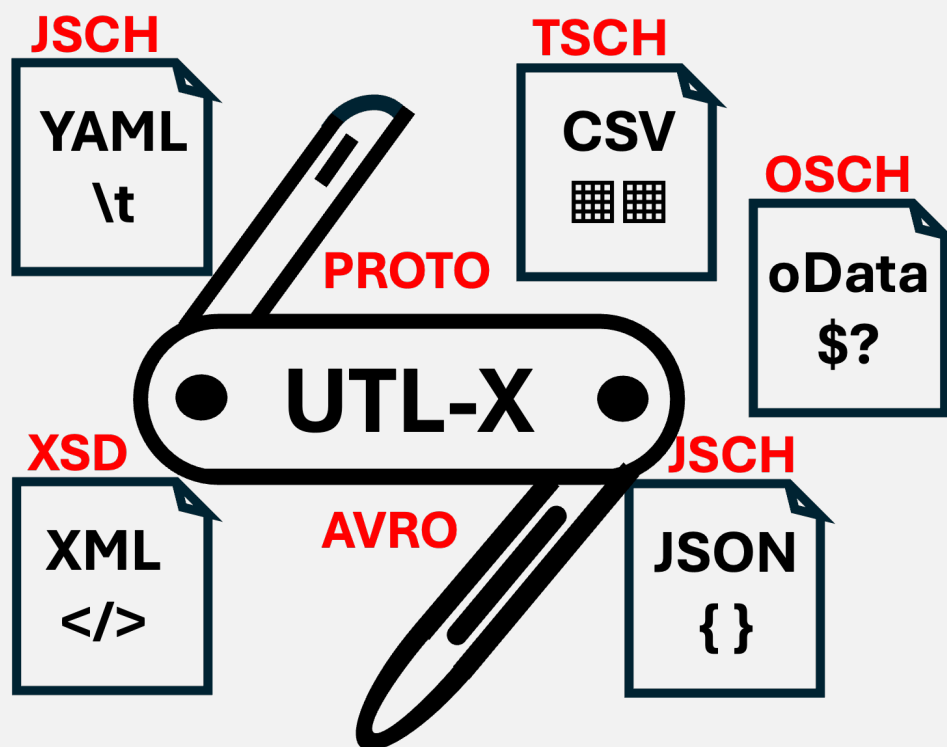


UTLX

Universal Transformation Language eXtended



UTLX for AI Token Reduction

A Lean & Six-Sigma Approach to Trimming Structured Input for LLMs

Ir. Marcel A. Grauwen

UTLX for AI Token Reduction — A Lean & Six-Sigma Approach to Trimming Structured Input for LLMs

Copyright © 2026 Ir. Marcel A. Grauven. All rights reserved.

Published by GLOMIDCO B.V., The Netherlands

First edition, 2026.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without the prior written permission of the author, except for brief quotations in reviews and critical articles.

UTLX is open source software. The language specification, CLI tool, and standard library are freely available at <https://github.com/grauven/utl-x>.

This is a companion article to **Many to One: The Theory of N:1 Data Mapping** and **UTLX: One Language, All Formats**. Where those develop the theory and teach the language, this one applies UTLX to a practical problem: reducing the token cost of structured data sent to language models.

Typeset with Typst in New Computer Modern.

About the Author

Ir. Marcel A. Grauwen is a Dutch software engineer and architect with over twenty-five years of experience in enterprise integration, data transformation, and middleware platforms.

He designed UTLX — one transformation language that works on all data formats, runs anywhere, and belongs to no vendor — and built its runtime, standard library, and the USDL schema-classification system. This article grew out of a practical question raised while deploying UTLX alongside LLM pipelines: how much of what we send a model is waste, and how much of that can UTLX remove safely?

Marcel holds an Ir. degree (Master of Science in Engineering) and is based in the Netherlands.

Thesis. The structured data we feed to language models is full of waste. Some of that waste is *pure* — the same information, encoded in a costlier shape — and can be removed deterministically with zero loss. The rest is *content* the model may or may not need, and can only be removed by **measuring**, the way a Six-Sigma process reduces variation: define, measure, analyse, improve, control. UTLX — format-agnostic, path-addressable, deterministic — is the right tool for both, and it already has most of the machinery.

Contents

1. Tokens Are the New Waste	5
1.1 How Tokenizers Count — BPE in One Minute	5
2. The Map of the Territory	5
3. The Easy Flip — Lossless Format Reduction	6
3.1 Why format choice moves tokens	6
3.2 The numbers (and their caveat)	6
3.3 The deterministic CSV-eligibility test	7
3.4 This is archetype detection — reuse, don't rebuild	7
3.5 Why the flip is safe	7
3.6 Lean the Input, Not Necessarily the Output	7
4. The Complex Reduction — Lossy Field Pruning	8
4.1 A different animal	8
4.2 Six Sigma for prompt pruning (DMAIC)	8
4.3 Measuring relevance — the methods	8
4.4 From relevance to a pruning mapping	9
4.5 Guardrails — the edge-case problem	9
4.6 The control loop	10
5. Key / Tag Abbreviation — A Second-Order Optimization	10
6. The Economics — Recurrence Is the Gate	11
7. Deployment — UTLX as a Gateway Transform	11
8. A Measurement Framework (Six Sigma, Operationalised)	11
9. Honest Limits & Risks	12
10. The Takeaway	12
11. Theoretical Grounding & Related Work	12
Bibliography	13

1. Tokens Are the New Waste

Every token sent to an LLM costs money and latency, and on a repetitive pipeline — an agent prompt fired thousands or millions of times — that cost compounds without bound. Yet the payloads we send are rarely lean: they carry verbose serialization, repeated keys, fields the model never reads, and structure that exists for machines, not for the model.

Two industrial disciplines name this problem precisely:

- **Lean** calls it *muda* — waste. Lean’s first move is to separate waste that can be eliminated *freely* (it adds no value) from reductions that require *judgement* (they might remove something needed).
- **Six Sigma** gives us the loop for the hard cases: **DMAIC** — Define, Measure, Analyse, Improve, Control — because when you cannot *prove* a reduction is safe, you *measure* it, and you keep measuring.

This article maps both onto UTLX. The pitch in one line: **UTLX turns a payload into the leanest representation the model still understands, deterministically where it can and by measurement where it must.**

1.1 How Tokenizers Count — BPE in One Minute

Everything that follows rests on one fact about how models read. An LLM does not see characters or words — it sees **tokens**, and you pay per token, on the input *and* the output. A *tokenizer* maps text to tokens, and modern ones are **subword** tokenizers, most commonly **BPE — Byte-Pair Encoding**. BPE starts from individual bytes and, over a huge training corpus, repeatedly merges the most frequent adjacent pair into a single token. The result is a vocabulary in which *common* strings — frequent words, frequent substrings — are a single token, while *rare or unusual* strings fragment into many small pieces. (GPT models use BPE variants like `tiktoken`; Claude and others use their own subword tokenizers of the same family. The exact merges differ; the principle does not.)

Five consequences drive the rest of this article:

- **Token count $\neq$ character count $\neq$ word count.** A *shorter* string can cost *more* tokens if it is unusual.
- **Common is cheap, ad-hoc is dear.** `preserve` may be one token; `prsrvr` may be three. This is why vowel-stripping and cryptic abbreviations (§5) often *cost* tokens instead of saving them.
- **camelCase splits.** Composite names like `preserveNullValues` typically break at the case boundaries into several tokens — what makes verbose keys expensive and *meaningful* abbreviation occasionally worthwhile.
- **Repetition repeats the cost.** Every occurrence of a key or tag is re-tokenized, so structure that repeats — JSON/YAML keys per record, XML’s paired tags — pays its token cost again and again. This is the heart of the format argument (§3).
- **Punctuation and whitespace are tokens too.** Braces, quotes, commas, indentation all count — which is why YAML beats JSON, and CSV beats both on the right shape.

Whenever a claim in this article seems counter-intuitive — *why would a shorter string cost more?* — the answer is always here: **the tokenizer rewards what it has seen often, not what is short.** If you take one tool away, take this: run your real field names and sample payloads through a token counter (`tiktoken` or your provider’s) and measure, rather than reasoning from character counts.

2. The Map of the Territory

There are three distinct reductions, and conflating them is the most common way to get this wrong. They differ in whether they lose information, how the reduction is derived, and how risky they are.

#	Technique	Loses info?	How it's derived	Risk	When worth it
1	Format flip (JSON→YAML→CSV)	No — same data, fewer tokens	Deterministic structural test	Low	Almost always
2	Field pruning (drop low-relevance fields)	Yes — a measured bet	Empirical (ablation / attention)	Medium-high	Recurring prompts only
3	Key/tag abbreviation (quantity→qty — <i>meaningful</i> ; not preserveNullValues→prs — <i>cryptic</i>)	Semantically	Heuristic + measure	Med (meaningful) / High (cryptic)	After pruning, on repetition-heavy data

The dividing line is **equivalence**. Technique 1 produces an *equivalent* payload — the bytes differ, the information is identical — so it is safe by construction. Techniques 2 and 3 *leave* the equivalence-preserving world: they change what the model sees, so they cannot be proven correct, only measured. Keep this line bright; it governs everything that follows.

3. The Easy Flip — Lossless Format Reduction

3.1 Why format choice moves tokens

Token cost is dominated by *repetition of structure*. The information in a record is its values; everything else — key names, braces, quotes, indentation — is overhead the model must still tokenize.

- **XML** is the most expensive: every field is wrapped in an *open and close* tag (<field>...</field>), so each key is tokenized roughly **twice** per record, plus angle brackets.
- **JSON** repeats every key name once per record, plus quotes, braces, and commas.
- **YAML** drops the quotes and braces but still repeats every key on every record; it wins on the *punctuation*, not the *keys*.
- **CSV** pays the key-name cost **exactly once** — the header row — and every subsequent record is just values and commas.

For an array of 100 objects with 8 fields each, YAML and JSON pay for `fieldName`: **100 times**; CSV pays for it **once**. That is the whole game.

3.2 The numbers (and their caveat)

On *flat, uniform, repeated* records — the CSV-friendly shape — measured on a representative sample reshaped as a flat array of records:

- **CSV beats JSON by ~37%.**
- **CSV beats YAML by ~32%.**

More generally, away from the CSV-eligible shape, the ranking holds in softer form: **YAML runs ~20–30% below JSON** (it sheds braces, brackets, quotes, and commas for indentation) and **~40–50% below XML** (whose open/close tag pairs tokenize each key roughly twice). So even when CSV does not apply, **YAML is the right default** for a token-bound payload, and JSON and XML are the formats to flip *away* from.

Compare that to the *nested* case (a wrapper object, sub-objects, mixed types), where YAML's edge over JSON is only modest and CSV either doesn't apply or needs flattening that reintroduces overhead (repeated path-prefix columns). The lesson:

CSV's advantage is structural, not universal. It is large and growing on uniform tabular data, and it shrinks to nothing — or reverses — on nested, sparse, or small data.

(These percentages are sample-dependent: they scale with field count, row count, and value width. Treat them as the *shape* of the answer, not a constant.)

3.3 The deterministic CSV-eligibility test

You do not want to guess case-by-case. A small static analyser — implementable as a UTLX pre-pass — decides eligibility per array subtree with five checks:

1. **Locate candidate arrays.** Walk the tree; find every repeated-element node with occurrences above a threshold (e.g. ≥ 3).
2. **Schema-uniformity check.** Collect the key-set (or child-element-set, for XML) of every item. CSV-eligible if all items share an identical key set — or a strict superset/subset relationship you’re willing to pad with empty cells — **and** no item has a nested object/array value (CSV cells must be scalar; nested values either disqualify, or must themselves be uniform and flattenable into dotted columns without cardinality blow-up).
3. **Sparsity check.** Compute the fraction of present (non-null) cells across all items. Below some **density threshold** (e.g. $> 85\%$), CSV wastes columns on empty cells and the saving collapses.
4. **Cardinality check.** CSV’s fixed cost is the header, paid once, so the advantage scales with row count. Below **~3–4 rows** the header isn’t amortised and JSON/YAML may win; above it, CSV’s advantage grows monotonically.
5. **Value-type check.** All values scalar, with no embedded newlines/commas/quotes demanding heavy escaping — escaping overhead eats the saving on free-text-heavy data.

Decision cascade:

```
for each array subtree:
  if passes all five checks → emit a flattening transform → CSV
  else                      → leave as-is for the YAML pass
whole-document default:    → YAML (beats JSON/XML for everything else)
fallback:                  → keep original (if the model needs it / for safety)
```

The result is *mixed-format*: the tabular parts of a document become CSV blocks, the irregular parts stay YAML, each labelled so the model knows what it’s reading.

3.4 This is archetype detection — reuse, don’t rebuild

“Is this array uniform and flat?” is **not a new problem for UTLX**. It is exactly the *structural archetype* classification already built for schema analysis: a **flat-list archetype** (one repeating entity, no nesting) is the CSV-eligible case; **header-detail**, **star**, and **deeply-nested** archetypes are not. The five checks above are a token-cost specialisation of the same type inference that drives the correspondence-set / strategy work (IF20). Implement the CSV test as a *pre-flight analysis pass over the existing schema-type taxonomy*, not a separate bolt-on.

3.5 Why the flip is safe

The flip is **deterministic** (the same payload always flips the same way), **lossless** (the parsed UDM tree is identical whichever format it serialises to — the flip is an *equivalence*), **reversible** (you can re-expand the model’s output back to the caller’s format), and **auditable** (you can diff the pre- and post-flip data and prove they’re equal). It is waste removal with no downside but one: the model must *understand* the format. LLMs read YAML and CSV well, but a CSV block needs a **header line** (the column legend) so the model knows what each column means — a small fixed cost amortised across the rows, and itself a reason the cardinality check matters.

3.6 Lean the Input, Not Necessarily the Output

A crucial asymmetry: token reduction targets the **input** payload, but the **output** format — what you ask the model to *generate* — should be chosen for *reliability*, not token thrift, and there the ranking inverts.

JSON is often the best output format despite costing more tokens, because it is unambiguous and models generate it validly far more often than YAML, whose whitespace sensitivity produces silent parsing errors in generated text. You pay for output tokens regardless; spending a few more for a reliably-parseable result is almost always right. So a well-tuned gateway is *format-asymmetric*: **flip the input lean (CSV/YAML), but request the output as JSON** (or a strict schema-constrained format). UTLX handles both directions — leaning the request, and re-expanding the JSON response into the caller’s format if needed.

4. The Complex Reduction — Lossy Field Pruning

4.1 A different animal

Dropping a field is **not** a format choice — it changes what the model sees. It leaves the equivalence-preserving world entirely, so it can never be *proven* safe; it can only be *measured*. The whole discipline of this section exists because of that one fact.

4.2 Six Sigma for prompt pruning (DMAIC)

- **Define** — the prompt, its inputs (the path-addressable structured record), and the output quality metric you refuse to regress (exact-match, embedding similarity, or task-specific score).
- **Measure** — for a representative, diverse sample, how much each field contributes to the output (Section 4.3).
- **Analyse** — aggregate per-path relevance; identify low-relevance paths and the long tail of edge-case-critical ones.
- **Improve** — emit a pruning mapping (a UTLX filter/select) that drops the low-relevance paths.
- **Control** — re-run the measurement periodically and on model/data change; chart output drift so a regression is caught, not discovered.

4.3 Measuring relevance — the methods

The right method depends on whether you have only an **API** or the **model internals**.

API-only (Claude, GPT, ...) — measure relevance empirically:

- **Ablation testing.** For each top-level field/path, run N variants with that field stripped, everything else held constant, and score the output drift vs the full-input baseline. The most practical approach for a repetitive pipeline: build the harness once, run it offline against historical inputs. A field whose removal doesn’t change the output is a reduction candidate.
- **Logprob-based attribution.** Where the API exposes logprobs, compare the log-probability of the expected output sequence *with* and *without* a field. A field whose removal barely shifts the logprobs of the correct answer is a candidate — a finer signal than exact-match, because it sees *near-misses* before they become wrong answers.
- **Perturbation / SHAP-style sampling.** Testing every field individually is expensive for nested structures. Instead, randomly mask *subsets* of fields across many runs and fit a linear attribution model (essentially SHAP for structured input) to estimate each field’s *marginal* contribution. This captures interactions (two fields redundant together) that one-at-a-time ablation misses.

Model internals (open-weight: Llama, Mistral, ...) — more direct signal:

- **Attention rollout / attention maps.** Aggregate attention weights across layers and heads to see which input tokens the *later* layers attend to most when generating the output. Tools like BertViz, or custom hooks in HuggingFace `transformers`, extract this. Caveat: raw attention is a *noisy* proxy for causal importance — attention is not explanation — so attention maps are best as a *hypothesis generator* that ablation then confirms, not a verdict on their own.

- **Gradient-based saliency / Integrated Gradients.** Compute the gradient of the output logits with respect to the input token embeddings (via Captum or similar) for a relevance score per input token. More faithful to causal contribution than raw attention, at higher compute cost.

Method	Needs internals?	Cost	Granularity	Signal quality
Ablation	No	High (N calls/ field)	Field/path	Direct, causal, coarse
Logprob attribution	No (needs logprobs)	Medium	Field/path	Direct, fine-grained
SHAP-style sampling	No	Medium-high	Field/path + interactions	Marginal, captures redundancy
Attention rollout	Yes	Low	Token	Proxy — confirm with ablation
Gradient saliency	Yes	Medium	Token	Faithful, per-token

For most UTLX users calling a hosted model, **ablation is the actionable default** (works regardless of provider, produces a clean structured-data answer); **logprob attribution** refines it where available; **attention/gradient** methods are for the open-weight case and are best treated as hypothesis-generators that ablation validates.

4.4 From relevance to a pruning mapping

This is where UTLX’s design pays off. Because UTLX already operates on **path-addressable** structured data (the same paths a correspondence rule uses), the per-path relevance scores map *directly* onto filter/select rules:

1. Take a representative sample of the repetitive inputs.
2. For each top-level path (XPath-/dot-style, exactly how UTLX addresses nodes), run the chosen measurement.
3. Score output drift vs the baseline; aggregate per path across the sample.
4. Paths below the relevance threshold become a UTLX transform that **elides them before the LLM call** — an *automatically-derived pruning mapping* rather than a hand-written one.

This closes a feedback loop: the relevance scores *are* correspondence rules; the analysis re-runs periodically to keep the mapping current as the model or data shape changes. The pruning mapping carries its own provenance — call it **ablation-derived** — distinct from a structurally or semantically authored rule, and explicitly *non-deterministic in origin* (it depends on the model and the sample), which is precisely why it must be re-validated rather than trusted forever.

4.5 Guardrails — the edge-case problem

The sharpest risk: “**doesn’t change *this* output**” is not “**never relevant.**” A field can be irrelevant for 95% of inputs and *critical* for an edge case. Aggregate statistics will happily prune it. Guardrails:

- **Large, diverse sample.** The measurement is only as good as the variety it saw; under-sample the tail and you prune the tail.
- **Never-prune declarations.** Fields involved in compliance, safety, or legally-relevant decisions are *declared* off-limits — never pruned by statistics, regardless of measured relevance. (Declare what the data cannot infer.)
- **Density-aware thresholds.** A rarely-present field that is decisive *when present* must be judged on its conditional impact, not its average.
- **Drift monitoring.** A control chart on output quality catches the day a pruned field starts mattering (a new data distribution, a model upgrade).

This is the token-cost twin of constraint soundness: a field critical 5% of the time is like a constraint that holds 95% of the time — you cannot clear it by averages alone.

4.6 The control loop

Re-run the study on a schedule and on every model or schema change. Pruning is not a one-time optimisation; it is a *controlled process* with a measured output and a tripwire.

5. Key / Tag Abbreviation — A Second-Order Optimization

Shortening verbose keys (`preserveNullValues` → `pnv`, `correspondence` → `corr`) is real but *second-order*: it helps less than pruning and carries a semantic risk pruning doesn't. Reason it through honestly, because both the dismissal and the enthusiasm are wrong.

It does help — and most where keys repeat. BPE tokenizers merge common subword patterns, but composite camelCase names break at the case boundary, so `preserveNullValues`, `rootElement`, or `correspondences` cost several tokens each. The saving *compounds with repetition*: a key that appears once is barely worth touching, but a tag repeated across an array — an XML element appearing 7 times, `open` + `close` = **14 occurrences** — is exactly where abbreviation pays, and is part of why XML loses to YAML so badly to begin with. Single-occurrence keys, and already-short common words (`id`, `type`, `from`, `to`), gain nothing — they are already one token.

The real tradeoff is meaning versus tokens, and it is a spectrum:

- *Cryptic codes* (`f`, `t`, `ty` for `from`, `to`, `type`) save the most but strip the semantic signal the model uses to understand the data — working directly against the relevance goal of §4. The model must guess what `f` means from context, adding noise, worst in zero-shot or low-context calls.
- *Common, meaningful abbreviations* (`addr` for `address`, `qty` for `quantity`) tokenize fine — they are frequent in training data — and keep enough meaning. These are the safe ones.

So abbreviation is lossy in a subtler way than pruning: pruning removes fields *measured* to be unneeded; cryptic abbreviation degrades the meaning of fields you are *keeping*. Prefer meaningful abbreviations; treat cryptic codes as a measured risk.

A popular method that backfires: removing the vowels. Disemvowelling — dropping the vowels (Dutch *klinkers*), so `preserveNullValues` becomes `prsrvNllvls` — is a classic human text-compression trick, and it is a *trap* for LLM tokens. It saves *characters* but usually *costs* tokens. A BPE tokenizer has learned merges for common, vowel-bearing words — `preserve` may be one or two tokens — but a vowel-stripped form is out-of-distribution and shatters into many small pieces (`pr` + `sr` + `v` ...), frequently *more* tokens than the original, not fewer. It also destroys meaning and introduces ambiguity (`ct` could be *cat*, *cot*, *cut*, or *act*). This is the cleanest illustration of the rule behind the whole section: **a shorter string is not a cheaper string**. An abbreviation helps *only* when it is itself a common, token-efficient string the tokenizer already merges — which ad-hoc vowel removal never is. If you abbreviate, reach for forms frequent in training data; never mangle characters.

Practical recipe — and the ordering matters:

1. **Prune first.** Cutting irrelevant fields (§4) almost always saves more than renaming the fields you keep, and carries no accuracy risk for correctly-identified-irrelevant fields. Abbreviate only *what survives the prune*.
2. **Test your tokenizer.** Run your actual key names through a tiktoken-style counter; long camelCase composites are the worst offenders and the best targets.
3. **Alias only the LLM-bound payload.** A UTLX transform maps verbose canonical names → a compact alias table *only* for the model's copy, leaving storage, logs, and downstream systems on the canonical names. Reversible, and the source schema is never touched.
4. **A/B test output quality.** The same ablation harness as §4, comparing accuracy and format-correctness before and after — never assume the saving is free of cost.

The verdict: a legitimate *third pass* after flip and prune — worth most on repetition-heavy, camelCase-heavy payloads, safe with meaningful aliases and a reversible table, risky with cryptic codes — and always smaller than the prune it follows.

6. The Economics — Recurrence Is the Gate

Both lossy techniques (and the *analysis* behind the lossless one) are **design-time investments amortised over run-time calls**. The arithmetic is simple and decisive:

worth it $\Leftrightarrow$ (per-call token saving $\times$ expected #calls) $>$ cost of the study + risk

- **One-off or low-volume calls** — the ablation study costs more than it will ever save. Don't measure; at most, apply the *free* lossless flip.
- **Recurring agent prompts** — the same template fired thousands or millions of times — fund a thorough study many times over. This is where pruning earns its place, and where the control loop pays for itself.

The lossless flip is cheap enough to apply *always* (its “study” is a deterministic test, not a measurement campaign). The lossy pruning is reserved for the recurring case. State this up front in any deployment so no one runs an ablation harness against a prompt used twice.

7. Deployment — UTLX as a Gateway Transform

The natural home is an **AI gateway** (Kong, or your own Open-M pipeline): a transform that sits in the request path between the caller and the model.

```

client → [AI gateway] → LLM provider
      |
      | request: parse → (prune mapping) → (format flip) → forward leaner
prompt
      |
      | response: parse model output → re-expand → return in caller's format

```

- **Design time (offline):** derive the format-flip rules (the five-check analyser) and, for recurring prompts, the pruning mapping (the ablation study). These are *artifacts* — versioned, reviewable, diffable.
- **Run time (per request):** the gateway applies the UTLX transform — prune, then flip — to each payload before forwarding, and optionally re-expands the response. UTLX is ideal here: format-agnostic, **deterministic** (same request $\rightarrow$ same leaner request, so the gateway stays cacheable and auditable), path-addressable, and fast at the edge.

This is the two-modes / three-times split made operational: the mapping is *authored* at design time and *executed* per message, with the model in the loop only during the offline study, never on the hot path.

8. A Measurement Framework (Six Sigma, Operationalised)

To run this as a process, not a guess, instrument four metrics and chart them:

- **Token reduction %** — pre- vs post-transform prompt tokens (the benefit).
- **Output drift** — exact-match rate, embedding similarity, and/or logprob delta vs the full-input baseline (the cost).

- **Cost & latency** — dollars and milliseconds saved per call (the payoff).
- **Quality on a held-out edge-case set** — a curated set of known-hard inputs the pruning must *not* regress (the guardrail).

The control chart on output drift is the heart of it: a flip is safe and needs only spot checks; a pruning mapping needs continuous monitoring, because the day the model or the data moves is the day a “safe” prune becomes a defect.

9. Honest Limits & Risks

- **Format comprehension.** Models read CSV/YAML well *with a header/legend*; without one, CSV is ambiguous. Always label format blocks and include the CSV header.
 - **Edge cases (pruning).** The central risk — covered by sampling diversity, never-prune declarations, and drift monitoring. Never prune by aggregate statistics alone.
 - **Model drift.** A model upgrade can change which fields matter; re-measure on every model change.
 - **Compliance & safety.** Some fields must never be removed regardless of measured relevance; encode that as policy, not preference.
 - **Escaping & free text.** CSV’s saving erodes on data with embedded delimiters/newlines; the value-type check guards this.
 - **Sample dependence.** All token-percentage figures scale with field count, row count, and value width — measure on *your* data.
-

10. The Takeaway

Lossless first, lossy by measurement, abbreviation last. Flip the format wherever the deterministic test says it’s safe — that’s free waste removal and it ships today, reusing the archetype machinery UTLX already has — leaning the *input* to CSV/YAML while still asking the model to *output* JSON for reliability. Prune fields only for prompts run often enough to fund the measurement, only with guardrails against the edge cases, and only as a *controlled* process you keep re-measuring. Abbreviate last, on what survives the prune, preferring meaningful aliases over cryptic codes and a reversible alias table over an edited schema. UTLX is the right transform for all of it because it is format-agnostic, path-addressable, and deterministic — the same three properties that make a mapping trustworthy make a prompt lean.

From a full payload, the leanest the model still understands. Many tokens, fewer.

11. Theoretical Grounding & Related Work

Token reduction is not a novelty; it is a meeting point of several mature fields. Naming them turns the recipes above into something with a foundation — and shows where UTLX’s contribution actually sits.

Compression and information theory — the lossless/lossy split itself. The distinction this article is built on is the classical boundary between *source coding* and *rate-distortion* (Shannon, 1948). The lossless format-flip is source coding: re-encoding identical information toward its **entropy** floor — which is why CSV’s removal of repeated keys has a hard limit, and why nothing compresses below the data’s information content. The lossy field-pruning is rate-distortion: trading *rate* (tokens) for *distortion* (output-quality loss), so an ablation study is empirically tracing a rate-distortion curve and the relevance threshold is an operating point on it. The pruning *objective* has a precise name — the

Information Bottleneck (Tishby, Pereira & Bialek, 1999; Tishby & Zaslavsky, 2015): compress input X into Z minimising $I(X; Z)$ while preserving $I(Z; Y)$ for output Y . A field’s relevance is an estimate of $I(\text{field}; \text{output})$, and pruning is the Information Bottleneck applied to a payload.

Interpretability and attribution — the measurement layer. Every relevance method of §4 is feature attribution from interpretable machine learning: **Shapley-value attribution** (Lundberg & Lee, 2017) underlies the SHAP-style sampling; **LIME** (Ribeiro et al., 2016) is the perturbation lineage; **Integrated Gradients** (Sundararajan et al., 2017) is the gradient-saliency method; **attention rollout** (Abnar & Zuidema, 2020) aggregates attention flow. The article’s caution — *confirm attention maps with ablation* — is the “Attention is not Explanation” debate made operational (Jain & Wallace, 2019; Wiegrefe & Pinter, 2019).

Prompt-compression systems — the applied prior art, and where UTLX differs. A fast-moving line compresses prompts directly: **LLMLingua** and its successors (Jiang et al., 2023–2024) drop low-perplexity tokens; **Selective Context** (Li et al., 2023) prunes by self-information; **Gist tokens** (Mu et al., 2023) and **AutoCompressors** (Chevalier et al., 2023) learn soft compressions; **Lost in the Middle** (Liu et al., 2023) studies where surviving content should sit. These compress *free text, statistically, token by token* — lossy, learned, and hard to audit. UTLX occupies the complementary niche: *structured data, at the path level, deterministically* — a provably-lossless flip and a measured, reviewable pruning mapping, where the compression “codebook” is a UTLX transform you can diff, version, and reverse. **Auditable structured-data compression, not statistical free-text compression.**

Tokenization — why §1.1 holds. BPE began as a data-compression algorithm (Gage, 1994) and entered NLP as subword units (Sennrich, Haddow & Birch, 2016); the cost asymmetries the article exploits — common strings cheap, ad-hoc strings dear — are properties of that construction.

Quality engineering — the process framing. The Lean / Six-Sigma loop is statistical process control (Shewhart, 1931; Deming, 1986) with a measure of robust design (Taguchi): the ablation harness with a drift control chart is SPC applied to prompt quality, and the recurrence gate is the economics of when measurement pays.

Where UTLX sits. UTLX is the *transformation* layer; this body of theory is the *measurement and decision* layer, and they compose. Path-addressability makes $I(\text{path}; \text{output})$ a per-rule score; determinism and reversibility supply the auditability the lossless half guarantees and the lossy half records; the rate-distortion view gives a principled stopping rule. It is the same move the companion theory of N:1 mapping makes — grounding an engineering practice in established theory, so the tools have somewhere to stand.

Bibliography

- Abnar, S. & Zuidema, W. (2020). Quantifying Attention Flow in Transformers. *ACL*.
- Chevalier, A., Wettig, A., Ajith, A. & Chen, D. (2023). Adapting Language Models to Compress Contexts (AutoCompressors). *EMNLP*.
- Deming, W. E. (1986). *Out of the Crisis*. MIT Press.
- Gage, P. (1994). A New Algorithm for Data Compression. *The C Users Journal* 12(2).
- Jain, S. & Wallace, B. C. (2019). Attention is not Explanation. *NAACL*.
- Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y. & Qiu, L. (2023). LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models. *EMNLP*. (See also LongLLMLingua and LLMLingua-2, 2024.)
- Li, Y., Dong, B., Guerin, F. & Lin, C. (2023). Compressing Context to Enhance Inference Efficiency of Large Language Models (Selective Context). *EMNLP*.

- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F. & Liang, P. (2023). Lost in the Middle: How Language Models Use Long Contexts. *TACL*.
- Lundberg, S. M. & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions (SHAP). *NeurIPS*.
- Mu, J., Li, X. L. & Goodman, N. (2023). Learning to Compress Prompts with Gist Tokens. *NeurIPS*.
- Ribeiro, M. T., Singh, S. & Guestrin, C. (2016). “Why Should I Trust You?”: Explaining the Predictions of Any Classifier (LIME). *KDD*.
- Sennrich, R., Haddow, B. & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units (BPE). *ACL*.
- Shannon, C. E. (1948). A Mathematical Theory of Communication. *Bell System Technical Journal* 27.
- Shewhart, W. A. (1931). *Economic Control of Quality of Manufactured Product*. Van Nostrand.
- Sundararajan, M., Taly, A. & Yan, Q. (2017). Axiomatic Attribution for Deep Networks (Integrated Gradients). *ICML*.
- Tishby, N., Pereira, F. C. & Bialek, W. (1999). The Information Bottleneck Method. *Allerton Conf. on Communication, Control, and Computing*.
- Tishby, N. & Zaslavsky, N. (2015). Deep Learning and the Information Bottleneck Principle. *IEEE Information Theory Workshop*.
- Wiegrefe, S. & Pinter, Y. (2019). Attention is not not Explanation. *EMNLP*.